Sparsh-Vaani

Post-OCR Setup: Existing Excel/TXT Files + ESP32 + Voice + Notifications

Important: continue from your existing OCR work

You have already worked on OCR using OpenCV and Tesseract and created an Excel/database file and a TXT file. Do not repeat the OCR installation unless the checks below show a missing package. Keep your existing files; this guide focuses on the next integration stage.

1. Open the existing project folder

```
cd ~/Sparsh-Vaani
ls -lah
```

If your Excel/TXT files are stored in another folder, use that folder path instead. To locate likely files:

```
find ~ -type f \( -iname '*.xlsx' -o -iname '*.csv' -o -iname '*.txt' \) 2>/dev/null | head -100
```

2. Check the OCR setup (no reinstall)

```
python3 -c "import cv2, pytesseract; print('OpenCV and pytesseract OK')"
tesseract --version
```

3. Install only the integration libraries

```
sudo apt update
sudo apt install -y python3-venv python3-pip espeak-ng
```

```
cd ~/Sparsh-Vaani
python3 -m venv --system-site-packages .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install pyserial requests smbus2 gpiozero flask openpyxl
```

Why these libraries?

Package	Purpose
pyserial	USB serial commands and status messages between Raspberry Pi and ESP32
requests	HTTP/API calls for an IoT notification service
openpyxl	Read/write .xlsx Excel medicine database from Python
smbus2	I²C communication for supported peripherals connected to Raspberry Pi
gpiozero	Raspberry Pi GPIO inputs/outputs (not ESP32 GPIO)
flask (optional)	Local browser-based status/control page
espeak-ng (system package)	Offline voice output

4. Test USB serial communication

```
python -m pip show pyserial
ls /dev/ttyACM* /dev/ttyUSB* 2>/dev/null
```

Connect the ESP32 using a USB data cable. The port may be /dev/ttyACM0 or /dev/ttyUSB0; use the port actually shown on your Pi. Close Arduino Serial Monitor or other apps using that port before running your Python program.

5. Test voice output

```
espeak-ng "Sparsh Vaani is ready. Please scan the medicine label."
```

6. Keep the existing Excel and TXT files

```
ls -lh *.xlsx *.csv *.txt 2>/dev/null
```

Use openpyxl if your medicine database is .xlsx. Use Python's built-in csv module if it is .csv, and built-in file handling for .txt. You do not need a separate library to read a plain TXT or CSV file.

7. Export your new Python environment packages

```
python -m pip freeze > requirements-integration.txt
```

This records pip-installed Python packages. It does not include OS packages installed with apt, such as Tesseract or espeak-ng.

8. Integration flow

Existing OCR program reads the label → Raspberry Pi looks up the medicine in the existing Excel/CSV/TXT data → Raspberry Pi sends a command over USB serial → ESP32 operates the selected compartment and reports sensor status → Raspberry Pi speaks the status and sends configured reminders/notifications.

Hardware and safety notes

• A passing-tablet sensor confirms that a tablet passed the chute; it cannot prove the patient swallowed it. Ask for explicit patient confirmation if required.
• Power servos from a suitable regulated external supply; connect grounds together. Do not power servos from the ESP32 3.3V pin.
• Blynk/email notifications need network access and configuration of the chosen service. Keep credentials out of public GitHub repositories.
• If your project folder or filenames differ, change the example path; do not rename or overwrite the files you already created.